

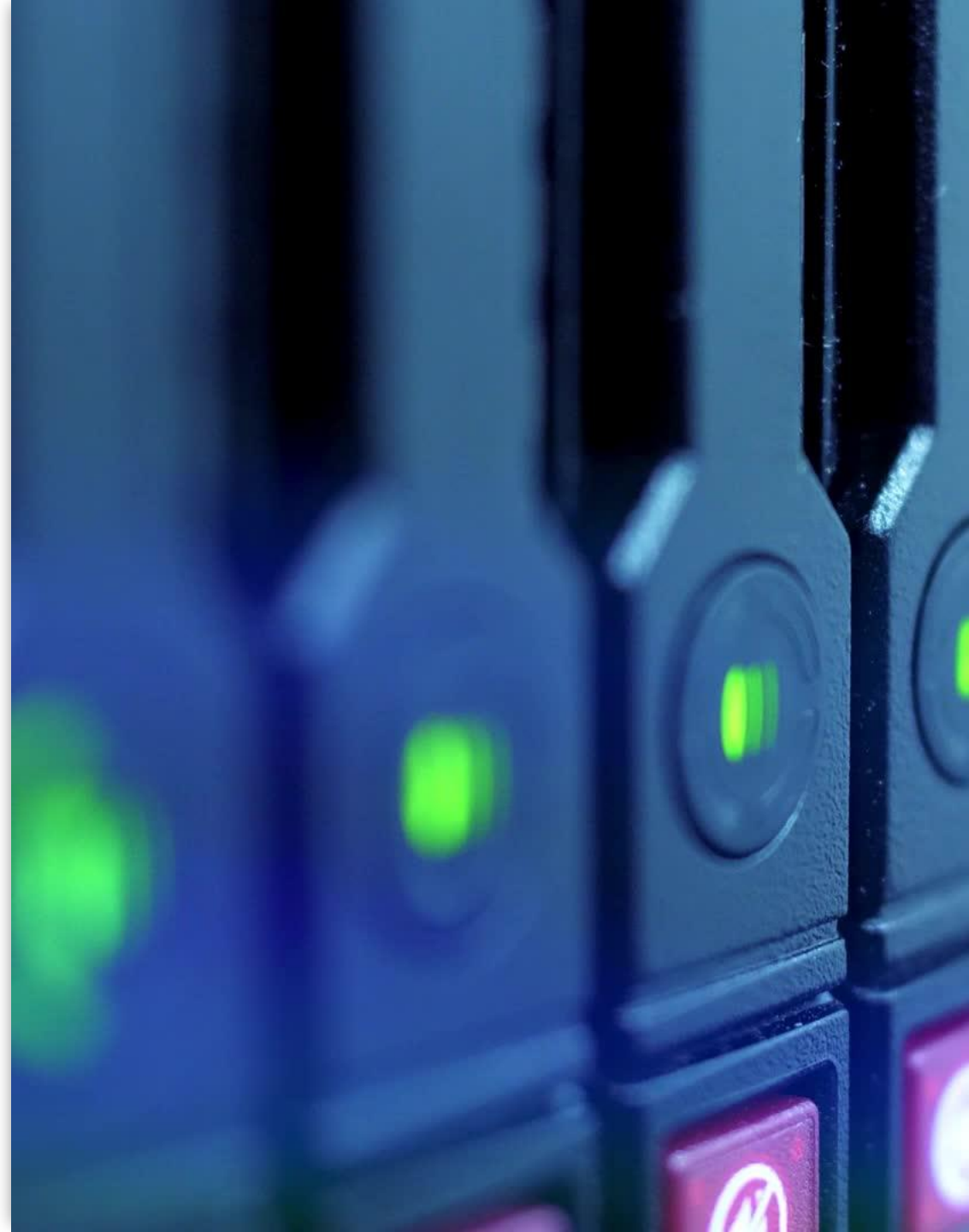


Going **big** with I/O

Anshul Singhvi

New age data access

- EO and climate data is often in the petabytes
- The data collected from a single satellite no longer fits on a laptop
- This is not a new problem!
- Solutions have been tried before:
 - Breaking up datasets into tiles
 - Storing data on the cloud
 - Allowing user access only behind special purpose servers (WCS, etc)



New age data access

- But all of these solutions come with drawbacks
- Multi file datasets mean the user has to know the structure of the dataset
- Large file storage on the cloud is hard to download, takes time
- Breaking up large files is expensive, cloud storage charges by number of keys
- Special purpose servers require maintenance and (importantly) user support – not guaranteed!





The dream

The dream

A dataset should only be represented by a URL;
that's all the user needs to know

Everything else (structure, access pattern,
chunking scheme, variables) should be read-
able from metadata

Users should be able to – but not **forced** to –
care about this!

I'm not talking about how to use the data – just
about how to get it so you *can* use it!

```
using Zarr, AWSS3
```

```
zopen("s3://its_live_data/velocity")
```

```
using Rasters, ZarrDatasets
```

```
Raster("s3://its_live_data/velocity")
```

```
using Kerchunk
```

```
Raster("reference://s3://noaa-er2/velocity")
```

```
using FilePathsBase
```

```
Path{S3}("s3://its_live_data/velocity")
```

How do we get there?

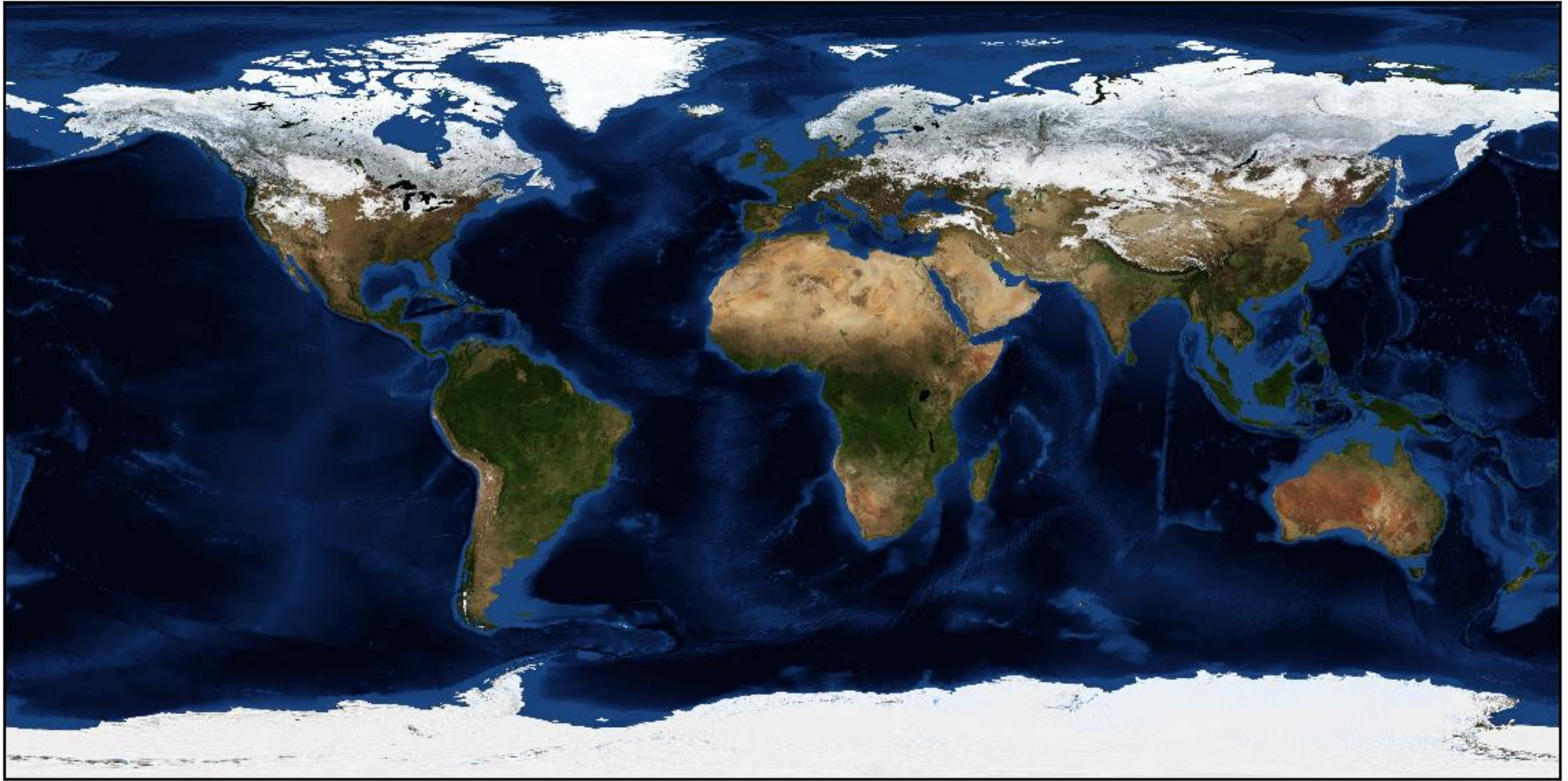
- Multi-file datasets can be unified using "sidecar" metadata files.
- Modern data storage methods make S3 hosting less expensive.
- Metadata is separated from data, so that it's immediately available
 - (this is the concept of cloud optimized data)
- Simple HTTP and S3 interfaces in most languages mean we can extend the idea of the "filesystem" to the Web, in a more robust way!
- Operations need to be asynchronous, to minimize time spent waiting for downloads.

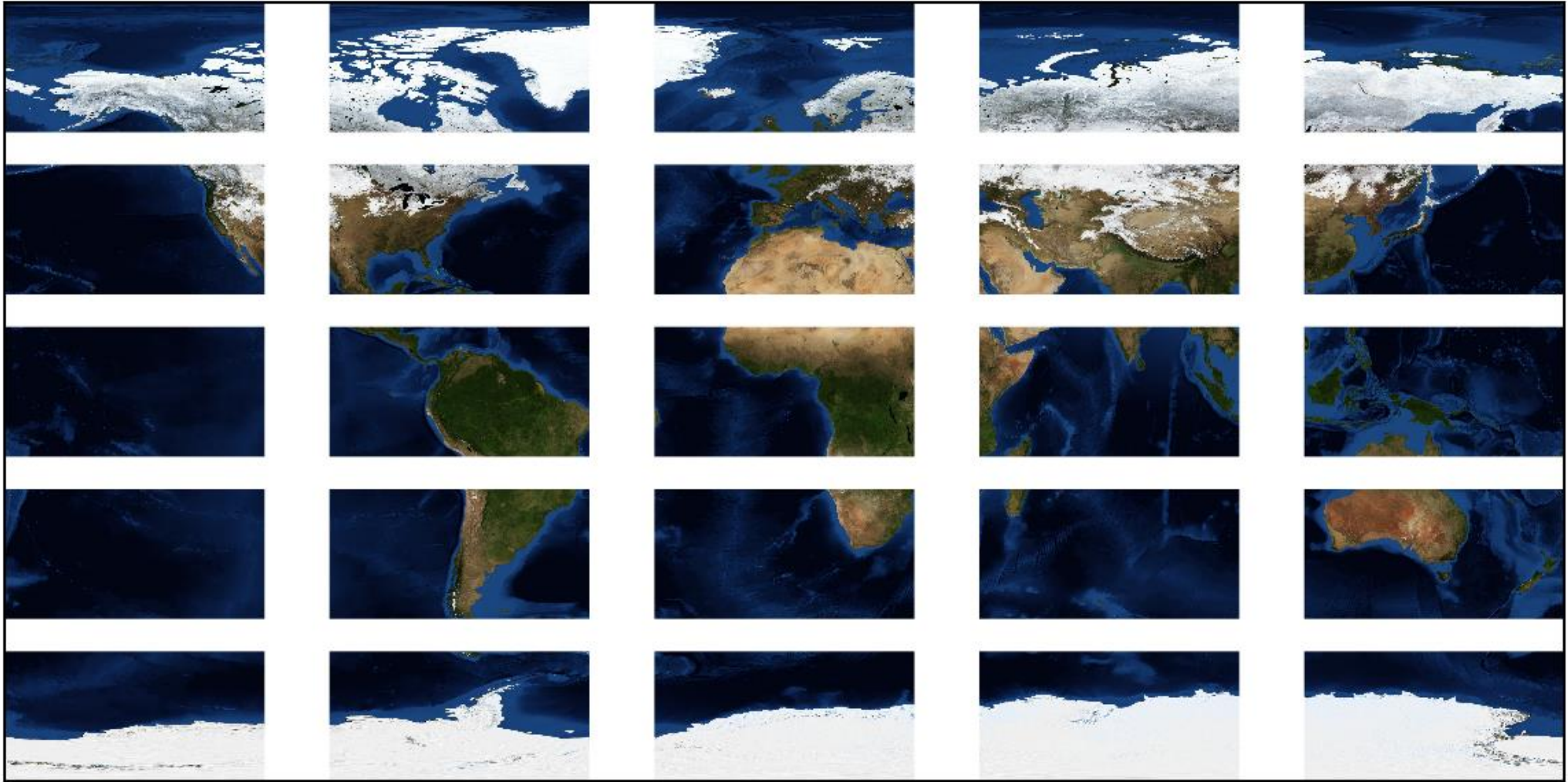


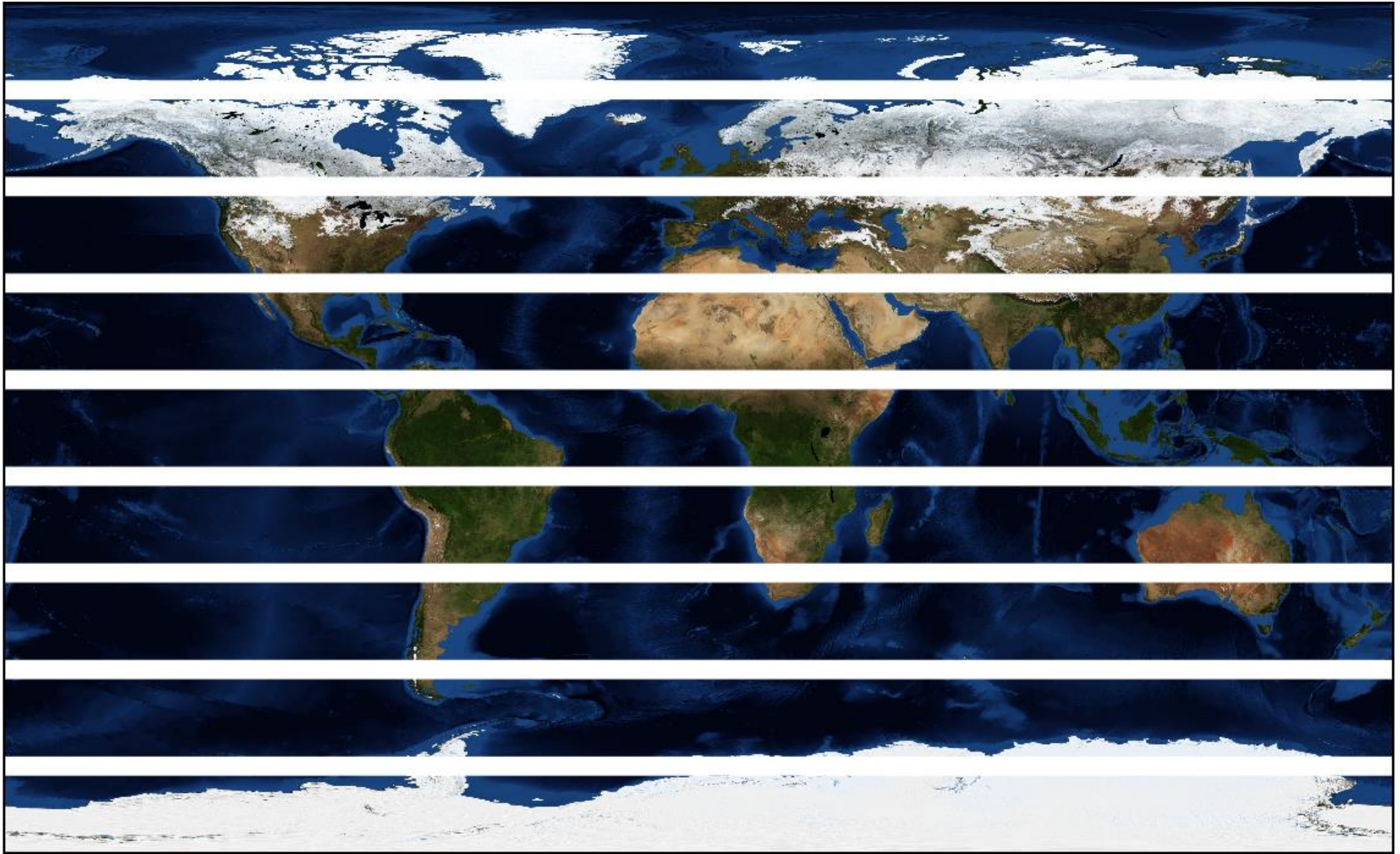
Optimizing access

Chunking

- At the end of the day, all data gets decomposed to a vector of numbers / bytes
- Storing a single 1TB byte vector is generally understood to be a bad idea
- So let's break it up into chunks!
- These chunks are basically hyper-rectangles.
- The chunk pattern dictates the optimal form of data access
- Chunking lets you (down)load only the data you need

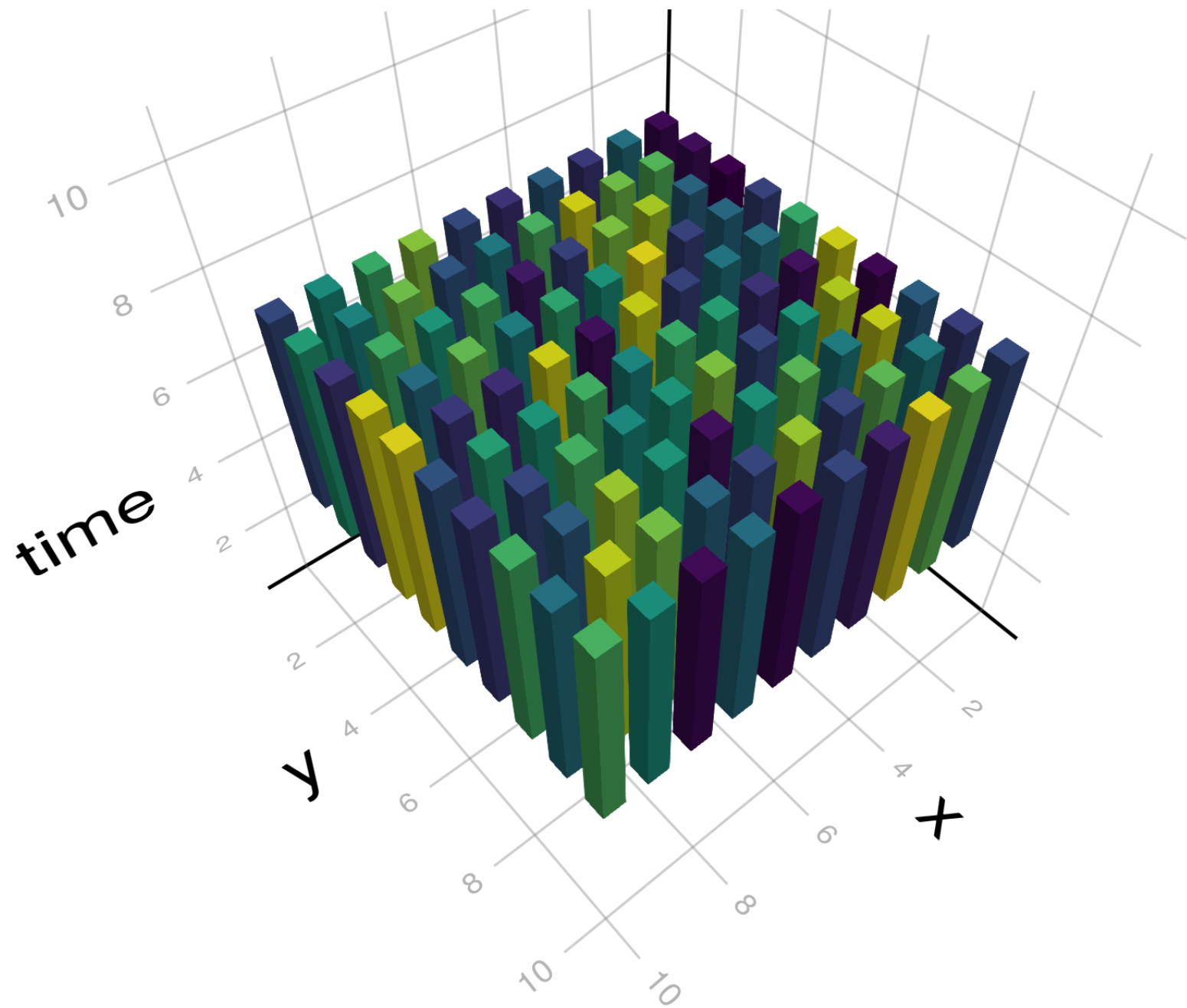


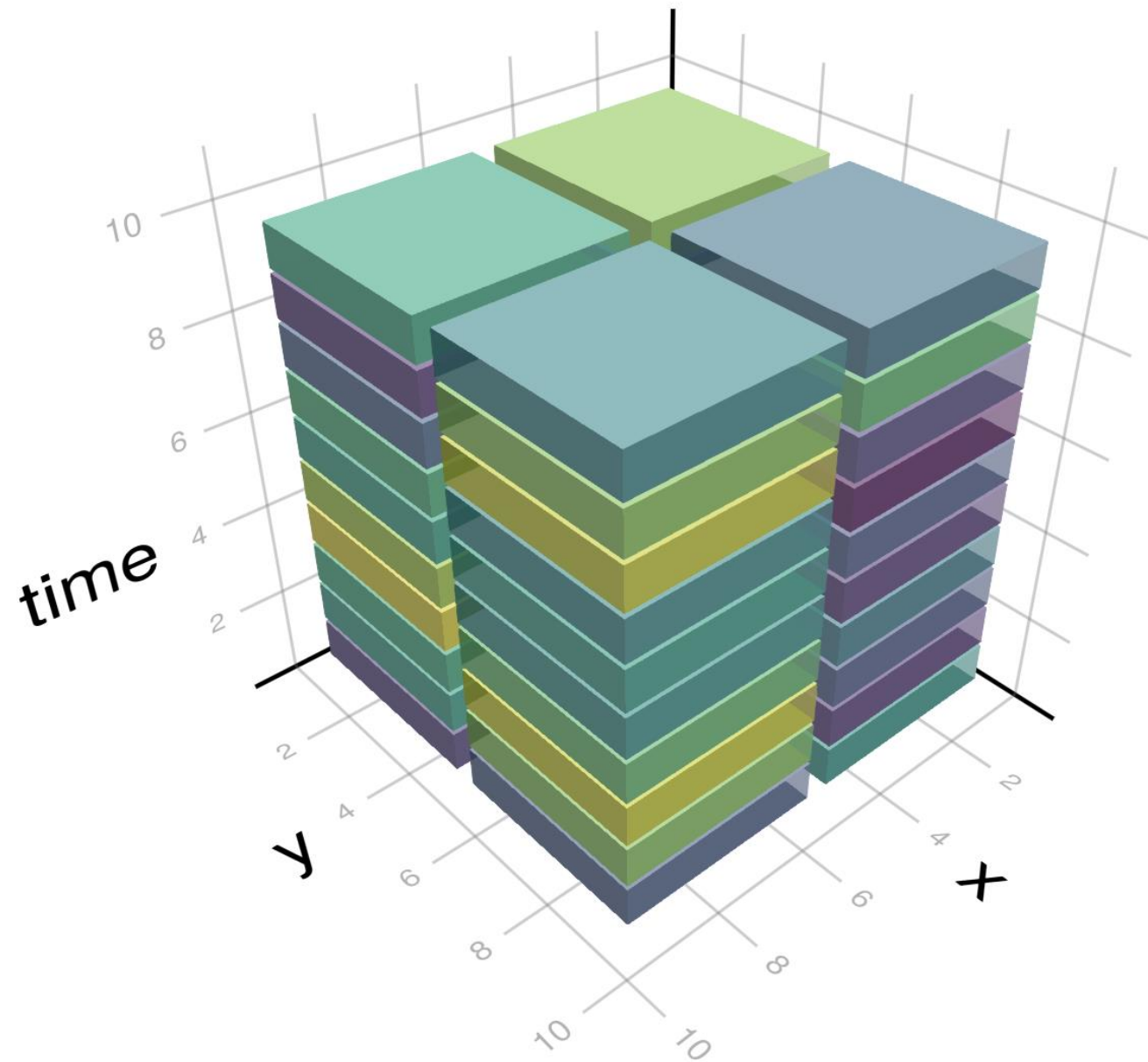




Chunking (in n-dimensions)

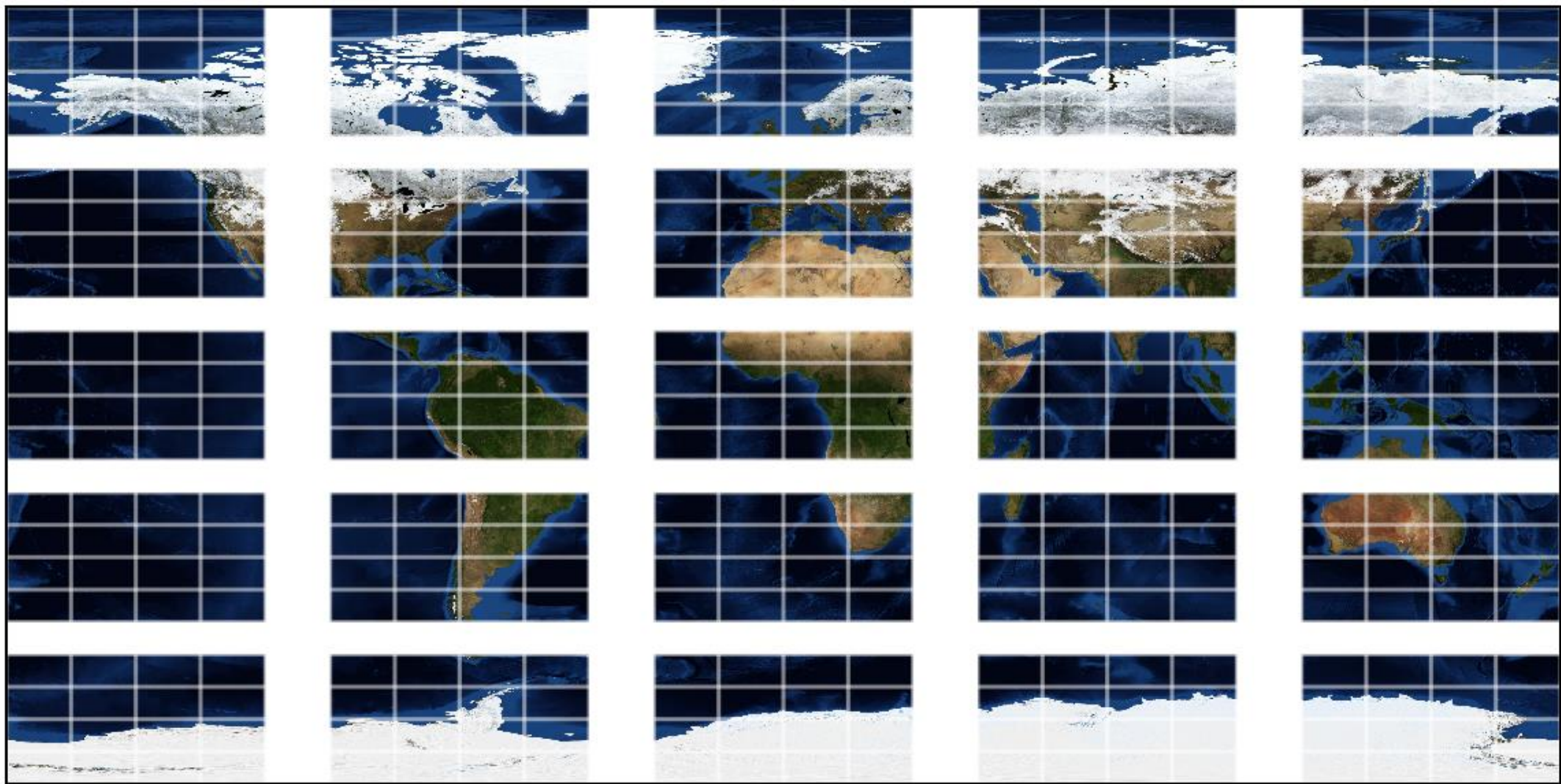
- But chunking is not restricted to 2D – it's also very interesting for 3D data
- If the users of your dataset are interested in long timeseries over a small spatial region, then you will chunk long in time and small in space
- If they are interested in global overviews at a single timestep – then you will chunk large in space and short in time

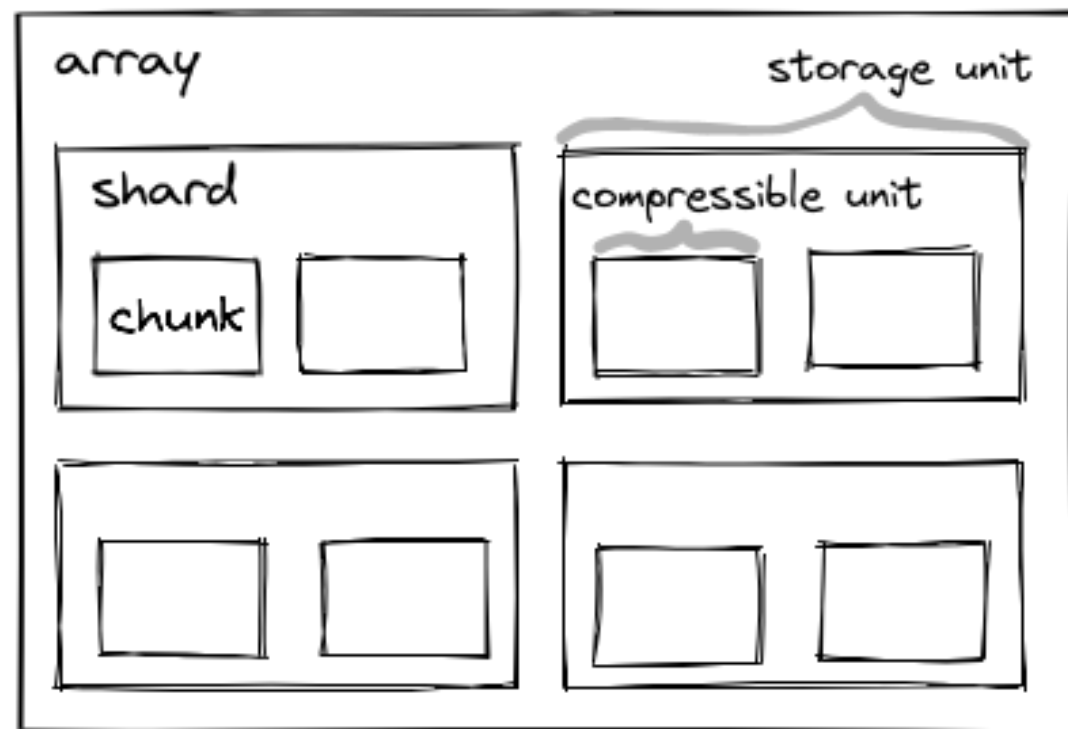


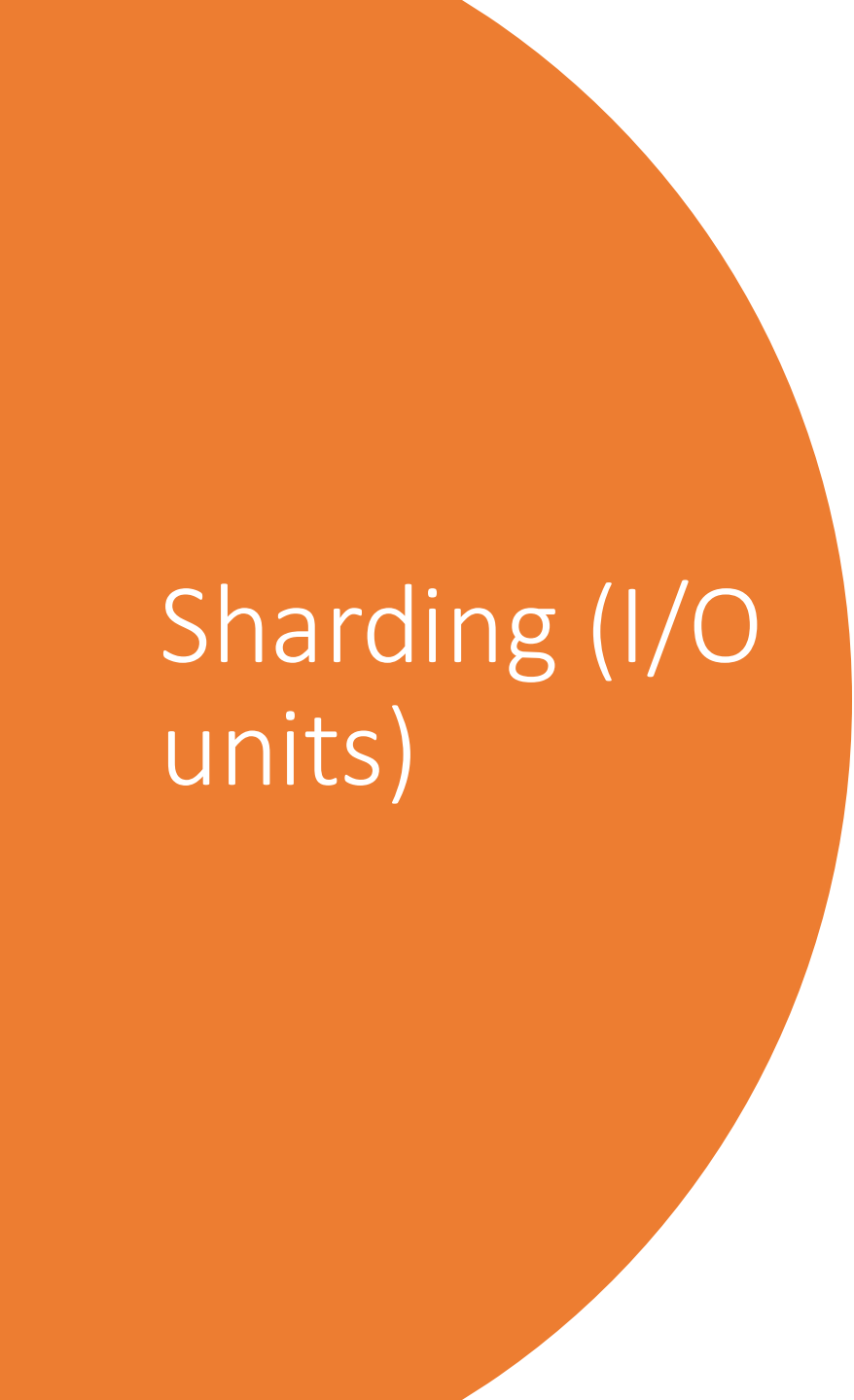


Sharding (I/O units)

- The more granular your chunks, the more of them you have
- That means that on S3, you have a lot of objects!
- In order to avoid this, we have the concept of *sharding* – concatenating multiple chunks into the same file
- This introduces quite a bit of complication – but reduces cost substantially
- It can also improve caching/download performance if used correctly







Sharding (I/O units)

- But this all sounds a bit familiar...



Sharding (I/O units)

- But this all sounds a bit familiar...
- This is the same as the multi-file dataset model
- But it's concatenated into a single logical dataset
- The user doesn't have to know about it!

Sharding (I/O units)

- But this all sounds a bit familiar...
- If each tile is an HDF5 file, then each tile is a “shard” where the chunks are the file’s chunks





Zarr - what and
why?



What is Zarr?

- Open format for storing large multidimensional data
- Flexible and extensible
- Multi-file by default – chunks stored in hierarchical directory structure
- Metadata stored as *separate files* as JSON
- Chunks can be compressed, transformed, etc – can even hold variable length strings, for example!

Why should we care?

Current storage solutions are complicated to read (HDF5, TIFF, NC4)

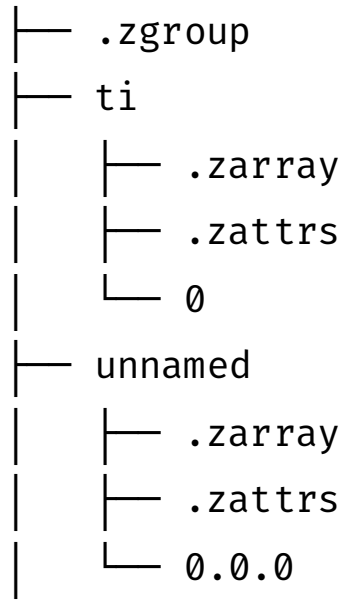
They almost always require some C library and encode information in binary

Zarr is simple and human readable, can be implemented easily in any language or platform

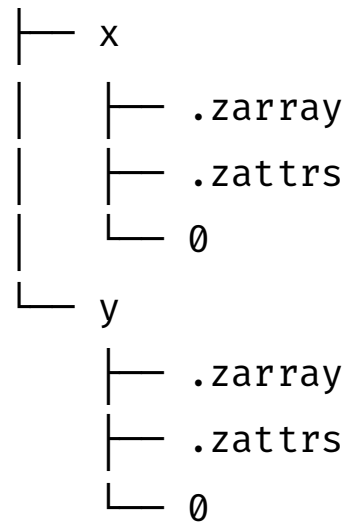
Filesystem abstraction means that it can even read inside zip files etc.

A sample Zarr structure

test.zarr



test.zarr



A sample Zarr structure

x/.zarray

```
{
  "chunks": [
    100
  ],
  "compressor": null,
  "dtype": "<i8",
  "fill_value": null,
  "filters": null,
  "order": "C",
  "shape": [
    100
  ],
  "zarr_format": 2
}
```

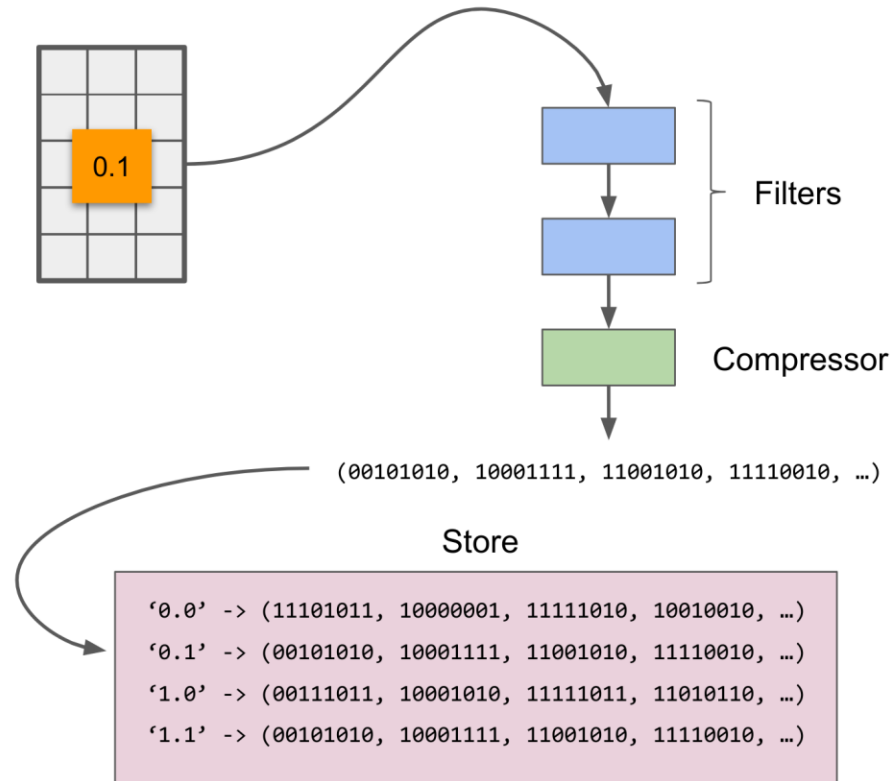
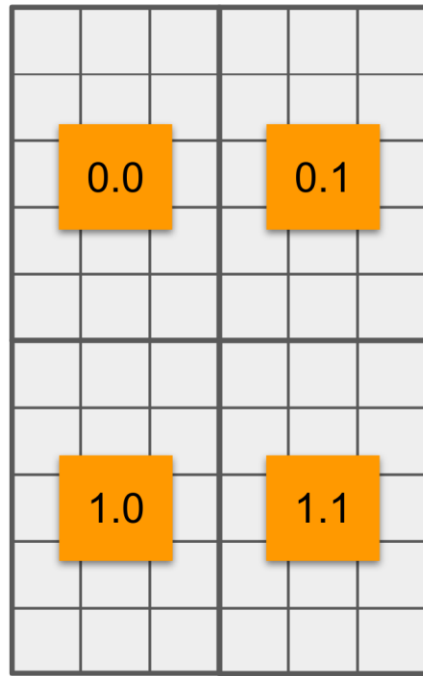
A sample Zarr structure

x/.zattrs

```
{
  "_ARRAY_DIMENSIONS": [
    "x"
  ],
  "axis": "X"
}
```

What is a chunk in Zarr?

E.g., array with shape (10, 6) and chunk shape (5, 3) has 4 chunks in a 2 by 2 chunk grid, with chunks identified by the keys '0.0', '0.1', '1.0', '1.1'.



The structure of a Zarr reader

- The core Zarr spec is very simple:
 - Indices -> chunks (handled by DiskArrays in Julia)
 - Chunk -> data (invert all compression and other transformations)
 - Chunk retrieval (store interface)

Kerchunk



What is Kerchunk?

- We love Zarr's interface, but we need to access non-Zarr data!
- How can we do this without stepping out of Zarr world?
- Chunks are chunks wherever you go...
- HTTP requests allow you to get specific byte ranges from files
- So why not create a “chunk manifest” mapping chunks to the actual binary chunks in HDF, TIFF, FITS, GRIB, etc files?

What is Kerchunk?

- Kerchunk is essentially a mapping
 - from chunk addresses `(0, 0, 0)`
 - to a file and byte range `(my_file.nc, 1204, 5502)`
- This mapping is stored in a JSON file
- Created by scanning all of the files in the dataset and computing the global position of each chunk
- Metadata is parsed out of the files and stored in the Kerchunk JSON file
- This allows millions of files to be accessed in one single dataset

What is Kerchunk?

- Kerchunk is essentially a mapping
 - from chunk addresses `(0, 0, 0)`
 - to a file and byte range `(my_file.nc, 1204, 5502)`
- This mapping is stored in a JSON file
- Created by scanning all of the files in the dataset and computing the global position of each chunk
- Metadata is parsed out of the files and stored in the Kerchunk JSON file
- This allows millions of files to be accessed in one single dataset

What is Kerchunk?

```
{
  ".zgroup": "{\n  \"zarr_format\": 2\n}",
  ".zattrs": "{\n  \"Conventions\": \"UGRID-0.9.0\n\"}",
  "x/.zattrs": "{\n  \"_ARRAY_DIMENSIONS\": [\n    \"node\"\n  ] ...",
  "x/.zarray": "{\n  \"chunks\": [\n    9228245\n  ],\n  \"compressor\": null,\n  \"dtype\":\n  \"x/0\": [\n    \"s3://bucket/path/file.nc\",\n    294094376,\n    73825960\n  ]
}
```

What is Kerchunk?

- The flat structure means that at some size of data, looking up the chunk dominates load time

VirtualiZarr

- VirtualiZarr is another way of expressing these chunk manifests
- It uses Kerchunk as a backend, but has the same hierarchical structure as Zarr (against Kerchunk's flat chunk list)



Solutions in Julia

- `DiskArrayEngine.jl` is a solution that optimizes computations over large, n-dimensional arrays
- `RangeExtractor.jl` (written by me) optimizes queries over small subsets of very large datasets
- `Kerchunk.jl` interprets Kerchunk reference files as Zarr stores.